

CO2-AT1- Analytical Problem Solving



Histogram Processing Sandboxes

Q1. Interactive Histogram Equalization & Saturation

Tools Used

- Google Colab – Used to execute the image processing program online.
- Python – Used for implementing the histogram equalization algorithm.
- OpenCV (cv2) – Used to read the image, convert it into grayscale, and apply histogram equalization.
- NumPy – Used for CDF calculation and finding saturation intensity values.
- Matplotlib – Used to display input/output images, histograms, and CDF graphs.

Task

To improve the quality of a low-contrast grayscale image by applying global histogram equalization and identify contrast saturation regions using CDF analysis.

Steps / Procedure

Step 1: Open Google Colab

1. Open Google Colab in the browser.
2. Create a New Notebook.
3. Install and import required libraries:
 - OpenCV
 - NumPy
 - Matplotlib

Step 2: Upload Input Image

1. Upload the low-contrast image into Google Colab using the file upload option.
2. Example file name:
 - input_image.jpg
3. Read the image using OpenCV.

Step 3: Image Preprocessing

1. Convert the uploaded image into grayscale format.
2. Display the original input image.
3. Analyze the intensity distribution.

Step 4: Generate Original Histogram

1. Calculate the histogram of the input image.
2. Display the intensity distribution graph.
3. Identify the limited intensity range of the low-contrast image.

Step 5: Apply Histogram Equalization

1. Apply histogram equalization using:
 - `cv2.equalizeHist()`
2. Enhance the contrast by redistributing pixel intensity values.
3. Save the enhanced image as:
 - `equalized_image.jpg`

Step 6: CDF Analysis and Saturation Detection

1. Calculate the Cumulative Distribution Function (CDF).
2. Identify flat regions in the CDF graph.
3. Detect intensity levels where contrast saturation occurs.
4. Create a saturation mask image.

Step 7: Display Results

Display:

- Original image
- Original histogram
- Equalized image
- Equalized histogram
- CDF graph
- Saturation region mask

Input Image

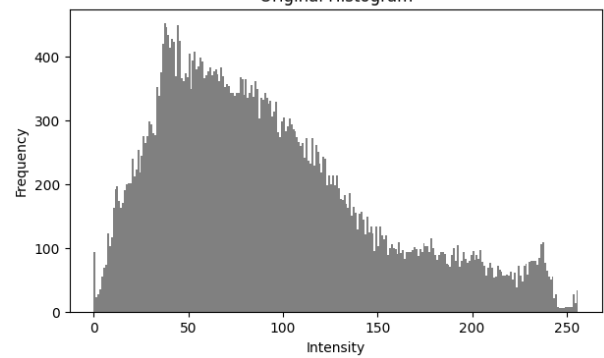


Output Images

Original Image



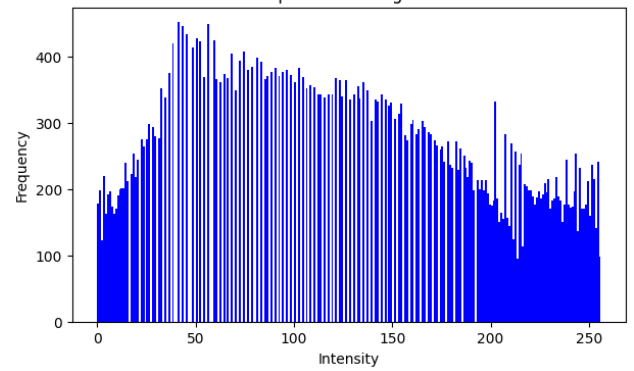
Original Histogram



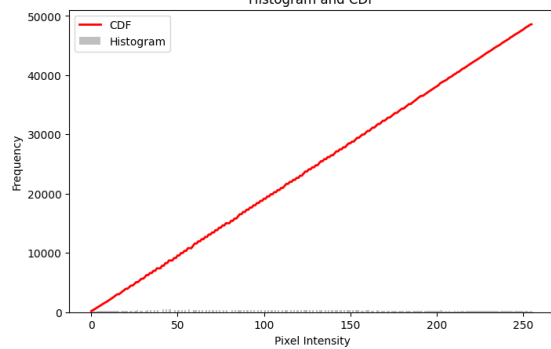
Equalized Image



Equalized Histogram



Histogram and CDF



Contrast Saturation Regions



Spatial Domain Filtering Sandboxes

Q3. Convolution Kernel Customization

Tools Used

- Google Colab – Used as the execution platform for running the image filtering code.
- Python – Used for implementing convolution operations.
- OpenCV (cv2) – Used for image processing and applying custom kernels.
- NumPy – Used for creating and modifying convolution matrices.
- Matplotlib – Used for displaying input and output images.

Task

To create a directional horizontal motion blur filter using a customized 3×3 convolution kernel while maintaining the original image brightness.

Steps / Procedure

1. Open Google Colab

- Create a new notebook.
- Import OpenCV, NumPy, and Matplotlib libraries.

2. Upload Input Image

- Upload the image into Google Colab.
- Read the image using OpenCV.

3. Create Custom Kernel

- Define a 3×3 convolution matrix for horizontal motion blur.

Kernel Matrix:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0.33 & 0.33 & 0.33 \\ 0 & 0 & 0 \end{bmatrix}$$

4. Apply Convolution

- Use OpenCV filter2D() function.
- Apply the custom kernel to the input image.

5. Display Output

- Show the original image and blurred output image.

- Save the output image.

6. Analyze Edge Changes

- Observe that vertical edges become smoother.
- Horizontal details are preserved.
- Image brightness remains unchanged because the kernel sum is 1.

Input Image



Output Image

Input Image



Horizontal Motion Blur Output



Q6. 2D Fast Fourier Transform (FFT) Phase Filtering

Tools Used

- Google Colab – Used as the execution platform for implementing FFT processing.
- Python – Used for frequency domain image processing.
- OpenCV (cv2) – Used for reading and processing images.
- NumPy – Used for FFT calculation and frequency manipulation.
- Matplotlib – Used for displaying images and frequency spectrum.

Task

To perform 2D Fast Fourier Transform (FFT) on an image and remove high-frequency components by applying frequency domain filtering. The effect is observed as a blur similar to a low-pass Gaussian filter.

Steps / Procedure

Step 1: Open Google Colab

1. Open Google Colab.
2. Create a new notebook.
3. Import required libraries:
 - OpenCV
 - NumPy
 - Matplotlib

Step 2: Upload Input Image

1. Upload a standard test image into Colab.
2. Example file name:
 - cameraman.jpg
3. Read the image using OpenCV.

Step 3: Convert Image to Frequency Domain

1. Convert the image into grayscale.
2. Apply 2D FFT using NumPy.
3. Shift the frequency components so that the low frequencies appear at the center.

Step 4: Apply Frequency Filtering

1. Create a frequency mask.

2. Remove high-frequency components far from the center.
3. Keep only low-frequency components near the origin.

Step 5: Convert Back to Spatial Domain

1. Apply inverse FFT (IFFT).
2. Generate the filtered output image.
3. Display the blurred image.

Processing

- Image is converted from spatial domain to frequency domain using FFT.
- High-frequency components are removed using a low-pass filter.
- Inverse FFT converts the filtered frequency data back into an image.

Input Image



Output Image

Input Image



FFT Low Pass Filter Output



Q8. Python OpenCV Edge Enhancement Sandbox

Tools Used

- Google Colab – Used as the execution platform for running image processing code.
- Python – Used for implementing edge detection techniques.
- OpenCV (cv2) – Used for applying Sobel and Laplacian operators.
- NumPy – Used for mathematical operations and gradient calculation.
- Matplotlib – Used for displaying input and output images.

Task

To enhance image edges using **Sobel X, Sobel Y, and Laplacian operators** on a grayscale bricks image. The Sobel kernel size is modified from **3×3 to 5×5** to observe changes in edge thickness.

Steps / Procedure

Step 1: Open Google Colab

1. Open Google Colab.
2. Create a new notebook.
3. Import required libraries:
 - OpenCV
 - NumPy
 - Matplotlib

Step 2: Upload Input Image

1. Upload the grayscale bricks image into Colab.
2. Example file name:
 - **bricks.jpg**
3. Read the image using OpenCV.

Step 3: Convert Image to Grayscale

1. Convert the input image into grayscale format.
2. Display the original image.

Step 4: Apply Sobel Operators

1. Apply **Sobel X** to detect vertical edges.

2. Apply **Sobel Y** to detect horizontal edges.
3. Calculate gradient magnitude using both X and Y derivatives.

Step 5: Apply Laplacian Operator

1. Apply the Laplacian filter to detect sharp intensity changes.
2. Generate the edge-enhanced output.

Step 6: Modify Sobel Kernel Size

1. Change Sobel kernel size from:
 - **3×3**
 - **5×5**
2. Compare the edge thickness and smoothness.

Step 7: Display Results

Display:

- Original image
- Sobel X output
- Sobel Y output
- Gradient magnitude image
- Laplacian output
- 5×5 Sobel output

Processing

- The image is converted into grayscale.
- Sobel operators calculate intensity changes in X and Y directions.
- Gradient magnitude combines both derivatives to show edge strength.
- Laplacian operator highlights sharp edges.
- Sobel kernel size is changed from 3×3 to 5×5 for comparison.

Input Image

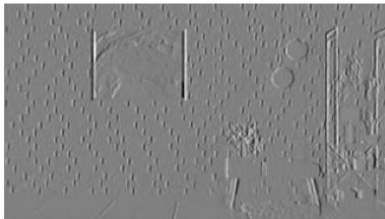


Output Image

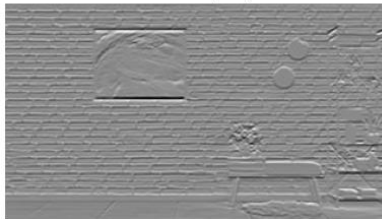
Original Grayscale Image



Sobel X (3x3)



Sobel Y (3x3)



Gradient Magnitude (3x3)



Laplacian Output



Gradient Magnitude (5x5)

